بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ

Allocation Functions

C / C++

Mohammed El Idrissi Laoukili
Instagram __elidrissii
Youtube IDRILOGIC

malloc()

Definition:

The “malloc” or “memory allocation” method in C is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form. It doesn’t Initialize memory at execution time so that it has initialized each block with the default garbage value initially.

Syntax:

```
ptr = (cast-type*) malloc(byte-size)
```

For Example:

```
ptr = (int*) malloc(100 * sizeof(int));
```

Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And, the pointer ptr holds the address of the first byte in the allocated memory.

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int *array, size, i;
    printf("Enter size: ");
    scanf("%d", &size);
    array = (int *)malloc(size * sizeof(int));
    // check if the memory created or not
    if (array == NULL)
    {
        printf("The creation failed!\n");
        printf("The memory not allocated.\n");
        exit(0);
    }
    else
    {
        printf("Memory created succsseful with malloc().\n");
        // get the elements
        for (i = 0; i < size; i++)
            array[i] = i + 1;
    }
    // display elements
    for (i = 0; i < size; i++)
        printf("%d\t", array[i]);
    return 0;
}
```

calloc()

Definition:

“calloc” or “contiguous allocation” method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. it is very much similar to malloc() but has two different points and these are:

- It initializes each block with a default value ‘0’.
- It has two parameters or arguments as compare to malloc().

Syntax:

```
ptr = (cast-type*)calloc(n, element-size);
```

here, n is the no. of elements and element-size is the size of each element.

For Example:

```
ptr = (float*) calloc(25, sizeof(float));
```

This statement allocates contiguous space in memory for 25 elements each with the size of the float.

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int *arr, size, i;
    printf("Enter size: ");
    scanf("%d", &size);
    // Dynamically allocate memory using calloc()
    arr = (int *)calloc(size, sizeof(int));
    // check if the memory has been successfully allocated by
calloc or not
    if (arr == NULL)
    {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else
    {
        printf("Memory successfully alloacated.\n");
        // Get elements
        for (i = 0; i < size; i++)
            arr[i] = i * 10 + 10;
        // Display elements
        printf("The elements of the array are:\n ");
        for (i = 0; i < size; i++)
            printf("%d\t", arr[i]);
    }
    return 0;
}
```

free()

Definition:

“free” method in C is used to dynamically de-allocate the memory. The memory allocated using functions malloc() and calloc() is not de-allocated on their own. Hence the free() method is used, whenever the dynamic memory allocation takes place. It helps to reduce wastage of memory by freeing it.

Syntax:

```
free(ptr);
```

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int *pointer, *pointer1;
    int size, i;
    size = 5;
    // Dynamically allocated the memory using malloc()
    pointer = (int *)malloc(size * sizeof(int));
    // Dynamically allocated the memory using calloc()
    pointer1 = (int *)calloc(size, sizeof(int));

    // check if the memory has been creaeted
    if (pointer == NULL || pointer1 == NULL)
    {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else
    {
        printf("Memory successfully allocated using malloc.\n");
        free(pointer);
        printf("Malloc memory successfully freed.\n");

        printf("\nMemory successfully allocated using calloc.\n");
        // Free the memory
        free(pointer1);
        printf("Calloc Memory successfully freed.\n");
    }
    return 0;
}
```

realloc()

Definition:

“realloc” or “re-allocation” method in C is used to dynamically change the memory allocation of a previously allocated memory. In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to dynamically re-allocate memory. Re allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

Syntax:

```
ptr = realloc(ptr, newSize);
```

where ptr is reallocated with new size 'newSize'.

If space is insufficient, allocation fails and returns a NULL pointer.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr;
    int size, i;
    size = 5;
    // Dynamically allocate memory using calloc()
    ptr = (int *)calloc(size, sizeof(int));
    // Check if the memory has been successfully allocated by malloc or not
    if (ptr == NULL)
    {
        printf("Memory doesn't allocated!\n");
        exit(0);
    }
    else
    {
        printf("Memory has been successfully allocated by calloc().\n");
        // Get the elements
        for (i = 0; i < size; i++)
            ptr[i] = i * 10 + 100;
        printf("The elements of the array are:\n");
        for (i = 0; i < size; i++)
            printf("%d\t", ptr[i]);
        // Get new size for the array
        size = 10;
        // Dynamically re-allocate memory using realloc()
        ptr = realloc(ptr, size * sizeof(int));
```

```
        // Memory has been successfully allocated
        printf("\nMemory successfully re-allocated using realloc.\n");
        // Get the new elements of the array
        for (i = 5; i < size; ++i)
            ptr[i] = i + 9;

        // Print the elements of the array
        printf("The elements of the array are: \n");
        for (i = 0; i < size; ++i)
            printf("%d, ", ptr[i]);

        free(ptr);

        // Print the elements of the array
        printf("\n\nAfte free() The elements of the array are: \n");
        for (i = 0; i < size; ++i)
            printf("%d, ", ptr[i]);
    }

    return 0;
}
```

اللَّهُمَّ صَلِّ علَى مُحَمَّدٍ وعلَى آلِ مُحَمَّدٍ، كما صَلَّيْتَ علَى إبْرَاهِيمَ وعلَى آلِ إبْرَاهِيمَ؛ إنَّكَ حَمِيدٌ مَجِيدٌ، اللَّهُمَّ بَارِكْ علَى مُحَمَّدٍ وعلَى آلِ مُحَمَّدٍ، كما بَارَكْتَ علَى إبْرَاهِيمَ وعلَى آلِ إبْرَاهِيمَ؛ إنَّكَ حَمِيدٌ مَجِيدٌ

وَإِذَا سَأَلَكَ عِبَادِے عَنِّے فَإِنِّے قَرِيبٌ ۖ أُجِيبُ دَعْوَةَ ٱلدَّاعِ ۖ إِذَا دَعَانِ ۖ فَلْيَسْتَجِيبُواْ لِے وَلْيُومِنُواْ بِيَ لَعَلَّهُمْ يَرْشُدُونَ ۖ